SCD Dbase Sorter - Project Documentation

Document: TECHNICAL_MANUAL.md

SCD Dbase Sorter - Technical Manual

**Vers

1. Project Overview

The *
Cell D
datab
introd

2. Core Features

* **M
The s

3. System Architecture

The s

4. Milestone 4: Superbot & Security Gateway

Milest

4.1 Companion Scanner App

* **F

Down

4.2 Staging API & Live Queue

* **A

final m

4.3 Structural Healing & Security Protocol

* **F

`-`, or

4.4 Visual Sync Box

* **F

Block

5. Chronological Approval Log

Detail

| Date | Task ID | Member | Role | Status | Message Body |

| :--- |

6. Deployment Instructions

1. Inst

---

7. Source Code Appendix

7.1 /home/team/shared/SCD_Dbase_Sorter/app.py

```pyth

```

import streamlit as st

impor

Add processor directory to path

sys.pa

Import team modules

from

??

Pag

??

SYS

Check for SSL (Conceptual check - in real production, check request headers)

st.w

Check for Master Database Password initialization

if not

return is_healthy, warnings

??

Ses

??

Sid

Health Check Status

health

st.sidebar.markdown("---")

SMTP Configuration

st.side

st.sidebar.markdown("---")

st.side

if st.sidebar.button("Save SMTP Settings"):

config

st.sidebar.markdown("---")

Data Paths Info

st.side

Hospital Config

st.side

st.sidebar.markdown("---")

Security / PII Section

st.side

Admin / Security Health Check

st.side

1. Key File Check

key_p

2. Audit Log Check

log_pa

3. Environment/Config Check

Che

if MASTER_DB_PATH and HOSPITALS_DIR:

st.side

4. Master DB Integrity (Encryption Check)

if os.p

Documentation Delivery Section

st.side

from mailer import _send_email

succe

st.sidebar.markdown("---")

External Discovery Section

st.side

st.sidebar.markdown("---")

st.side

??

Mai

====== STEP 1: File Upload ======

st.hea

Check if Master DB already exists to guide the user

maste

uploaded_file = st.file_uploader(

uploa

File Password (for encrypted files) ? only show if checkbox is selected

uploa

if uploaded_file is not None:

Sav

st.session_state.uploaded_filename = uploaded_file.name

st.suc

Preview the uploaded file

col1, c

st.dataframe(preview_df, use_container_width=True)

st.cap

with col2:

st.met

====== STEP 1.5: Manual Mapping & Teaching ======

st.hea

Check if this is a new file

if st.se

try:

Get

Auto-detect initial mapping

auto_

st.markdown("""

Revie

Create a dictionary to hold user selections

user_

We'll display it in a table-like format with dropdowns

cols =

Dropdown for master heading

defau

idx = 0

if defa

selected = st.selectbox(

f"Map

if st.button("? Confirm & Teach System", type="secondary"):

Upd

if source_header:

Tea

st.session_state.custom_mapping = user_mapping

st.ses

except Exception as e:

st.erro

====== STEP 2: Process & Sort ======

st.hea

process_disabled = not st.session_state.mapping_confirmed

if st.button("? Process & Sort", type="primary", use_container_width=True, disabled=process_disabled):

with s

Step B: Process (append to master, generate hospital sheets)

maste

Store in session state

st.ses

st.success(f"? Processing complete! {len(mapped_df)} records processed.")

except Exception as e:

st.erro

Clean up temp file at the end of every run to avoid storage leak

if os.p

else:

st.info

====== STEP 3: Data Overview & Visualization ======

st.hea

if st.session_state.processing_complete and st.session_state.master_df is not None:

maste

Use tabs for different views

tab_o

with tab_overview:

col1, c

st.markdown("---")

Records per Hospital chart

if "Hos

Bar chart using st.bar_chart (built-in, no extra deps)

chart_

Show numeric table

col1, c

Records per Year chart

if "Yea

chart_year = year_counts.set_index("Year")

st.bar_

with col2:

st.data

Cross-tab: Hospital x Year

if "Hos

Show PII security note

if "Pat

with tab_hospitals:

st.sub

if selected_hospital:

hosp_

Show validation status breakdown

if "Val

with tab_raw:

st.sub

elif os.path.exists(MASTER_DB_PATH):

Try

pro

====== STEP 4: Email Actions ======

st.hea

if st.session_state.processing_complete and st.session_state.master_df is not None:

maste

if "Hospital" in master_df.columns:

hospit

Option to email all hospitals at once

col_bu

Export hospital data to temp file for attachment

safe_

result = send_validation_request(hospital, hosp_data, attachments=[temp_attach])

result

st.write("**Validation Request Results:**")

st.dat

with col_bulk2:

if st.bu

safe_name = str(hospital).strip().replace(" ", "_")

temp_

result = send_finalized_data(hospital, hosp_data, attachments=[temp_attach])

result

st.write("**Finalized Data Send Results:**")

st.dat

st.markdown("---")

st.sub

Individual hospital email controls

col_h

SCD Dbase Sorter - Project Documentation

col_info1, col_info2 = st.columns(2)

with c

if st.button("? Send Now", type="primary"):

if sele

if email_type == "Validation Request":

succe

if success:

st.suc

else:

st.info

Show existing database data for emailing, even if no new data was processed

if os.p

if st.button("? Send Validation Request"):

hosp_

??

Foc

from discovery_api import lead_initiate_request, get_final_discovered_df

from d

Master DB Password Store (simple hashed password for authorization)

MAST

sha25

def verify_master_db_password(password):

"""Ver

Session state for password authorization

if "disc

====== PAGE: Discovery Initiation ======

if disc

tab_single, tab_bulk, tab_csv = st.tabs(["Single Recipient", "Bulk (Multi-Line)", "Bulk (CSV Upload)"])

with tab_single:

with s

```
with col_help:
                                                                                                st.sub


with tab_bulk:
                                                                                                st.sub


st.code("recipient1@example.com, 5551234567\nrecipient2@example.com, 5557654321", language="text")

bulk_text = st.text_area(
                                                                                                "Recip




col_bcc, col_bph = st.columns([1, 3])
```

with c

if st.button("? Send Bulk Discovery Requests", type="primary", use_container_width=True):

if not

for i, line in enumerate(lines):

parts

if parse_errors:

st.war

if recipients:

with s

SCD Dbase Sorter - Project Documentation

```
results_df = pd.DataFrame(results)[["email", "token"]]
results

with tab_csv:
st.sub

csv_file = st.file_uploader("Choose CSV file", type=["csv"], key="bulk_csv_upload")
if csv_

if 'email' in csv_df.columns and 'phone' in csv_df.columns:
valid =

if st.button("? Send From CSV", type="primary", use_container_width=True):
recipi

with st.spinner(f"Sending {len(recipients)} discovery requests..."):
from c
```

====== PAGE: Request Tracking ======

elif dis

Load all requests from the discovery service

all_re

if not all_requests:

st.info

Check expiry

expire

SCD Dbase Sorter - Project Documentation

status = req.get("status", "Unknown")

if is_e

tracking_data.append({

"Toke

df_tracking = pd.DataFrame(tracking_data)

Summary metrics

col_m

st.markdown("---")

st.sub

Color the status column

def st

df_display = df_tracking.copy()

df_dis

st.dataframe(df_display, use_container_width=True)

Show raw request details on expand

st.mar

Find full token

full_to

if full_token and full_token in all_requests:

st.jso

Expand/collapse controls

col_re

====== PAGE: Recipient Portal ======

elif dis

Memorization / Loading Master DB context

if os.p

Session state for recipient flow

if "rp_

Step 1: Identity & OTP

if not

with st.form("rp_otp_init"):

email

if st.session_state.rp_otp_sent:

otp =

else:

Ver

tab_computer, tab_email = st.tabs(["? 1. Computer Search", "? 2. Email Search"])

with tab_computer:

st.hea

with tab_email:

st.hea

if st.button("? Submit All Discovered Files to Lead", type="primary"):

st.suc

st.markdown("---")

st.info

====== PAGE: Dashboard (default - show discovery expanders at bottom) ======

else:

Step A: Initiate Discovery Request

col_re

if st.button("? Send Discovery Request", type="secondary"):

if recip

st.markdown("---")

Step B: Pending Discovery Requests

st.sub

Real data from discovery requests

all_re

if tracking_list:

st.dat

st.markdown("### ? Review Scanned Data")

Sel

if result_tokens:

select

```
if selected_token_long:
                                                                    req =


final_discovered_df = get_final_discovered_df(selected_token_long)

if not final_discovered_df.empty:
                                                                    st.writ
```

```
st.markdown("---")
```

Step C: Password Authorization Gate

```
                                                                    st.sub

if not st.session_state.discovery_authorized:
                                                                    st.war

disc_password = st.text_input(
                                                                    "Enter


if st.button("? Authorize", type="primary", use_container_width=True):
```

if verif

st.markdown("---")

Step D: Append / Update Operations

st.sub

append_disabled = not st.session_state.discovery_authorized

col_append, col_update = st.columns(2)

with c

master_result = process_new_data(discovered_df)

st.suc

with col_update:

if st.bu

====== Discovery Log Section ======

with s

====== Visual Sync Box ======

st.mar

Session state for sync box

if "syn

Box container

box =

col_b1, col_b2, col_b3, col_b4 = st.columns(4)

with c

File cards with shield icons

for f in

Export button

if st.bu

elif st.session_state.sync_box_exporting:

Ani

if not queue_files:

All

total = len(st.session_state.sync_box_files)

done

Show remaining files with a "disappearing" effect

for idx

Simulate file processing

impor

Show files removed so far

if st.se

progress_bar.progress(100, text="Export complete!")

status

Clear state and vanish

st.ses

elif st.session_state.sync_box_exported is True:

Box

st.markdown("---")

====== Recipient Landing Page ======

with s

```
recipient_token = st.text_input(
                                                                    "Disco

if recipient_token:
                                                                    disc_r

st.markdown("---")
                                                                    st.sub

discovered_file = st.file_uploader(
                                                                    "Uploa

st.markdown("**? Encrypted File Support**")

if "recipient_cached_password" not in st.session_state:
                                                                    st.ses

if st.session_state.recipient_password_verified:
                                                                    st.info
```

effective_password = st.session_state.recipient_cached_password

if not

if discovered_file:

col_fn

if st.button("? Submit Discovery Files", type="primary", use_container_width=True):

with s

ext = os.path.splitext(discovered_file.name)[1].lower()

succe

if ext in ['.xlsx', '.xls']:

from r

if not mapped_df.empty:

if effe

master_hashes = set()

if os.p

missing_count = 0

if 'Pati

st.success(f"? File processed! {len(mapped_df)} records ({missing_count} new).")

if miss

```
elif ext == '.docx':
                                                                                from v
```

```
elif ext in ['.doc']:
                                                                                st.war
```

```
if success:
                                                                                updat
```

```
if os.path.exists(tmp_path):
                                                                                os.unl
```

```
st.markdown("---")
                                                                                st.sub
```

st.caption("SCD Dbase Sorter | Built with Streamlit | Team SCD Dbase Sorter")

```

### 7.2 /home/team/shared/SCD_Dbase_Sorter/processor/mapping.py

```pyth

from sanitization import sanitize_dataframe

from l

## Standard Master Headings

MAST

ALIAS_FILE = "/home/team/shared/SCD_Dbase_Sorter/data/config/aliases.json"

def load_aliases():

"""Loa

def save_new_alias(master_heading, new_alias):

"""Sav

os.makedirs(os.path.dirname(ALIAS_FILE), exist_ok=True)

with o

def _get_excel_data(file_input, password=None):

"""

```
def levenshtein_distance(s1, s2):
                                                                    if len(
```

```
def find_master_match(header_text, use_fuzzy=True):
                                                                    """Che
```

```
clean_text = str(header_text).strip().lower()
```

Check exact/case-insensitive master headings

```
                                                                    for ma
```

Check aliases

```
                                                                    aliase
```

Fuzzy Matching (Milestone 4)

if use

Check against master headings

for ma

Check against aliases

for ma

if best_match:

Hea

return None

def get_column_mapping(file_input, password=None):

"""

num_rows = df_scan.shape[0]

num_

best_row_idx = 0

best_

Milestone 4: Search up to Row 10

for ro

for col_idx in range(num_cols):

val =

Heuristic: Row 1 & 2 are special (often combined)

But

Check 30% threshold for deep scan rows (as per Sanitization Protocol 3.2)

if row_

If no matches found in deep scan, try combined Row 1 & 2 logic from before

if max

match = find_master_match(r1) or find_master_match(r2)

if mat

return best_mapping, best_row_idx

CONFIG_PATH = "/home/team/shared/SCD_Dbase_Sorter/data/config/hospital_emails.csv"

def load_hospital_config():

"""Loa

def mask_pii_data(df):

"""

```
def _mask_string(val):
                                                                                                    if not i
```

```
def load_and_map_data(file_input, password=None, custom_mapping=None):
                                                                                                    """
```

```
Args:
                                                                                                    file_in
```

```
excel_data = _get_excel_data(file_input, password)
```

Rea

Rename columns based on mapping

df_ma

Milestone 4: Record new aliases if fuzzy matching was used

We

excel_data_orig = _get_excel_data(file_input, password)

try:

for col_idx, master_name in mapping.items():

if col_

Add default values for missing master columns

for ma

Apply PII masking

df_ma

Apply Sanitization

from s

Set default Validation_Status if missing

if 'Vali

Set Date_Added

df_ma

file_label = file_input if isinstance(file_input, str) else "file-like-object"

audit_

Fill Validator_Email and Hospital_Email from config if missing

config

Fill missing Validator_Email

if 'Vali

Fill missing Hospital_Email

if 'Em

return df_mapped

```

### 7.3 /home/team/shared/SCD_Dbase_Sorter/processor/sorter.py

```pyth

MASTER_DB_PATH = "/home/team/shared/SCD_Dbase_Sorter/data/master/Master_Database.xlsx"

HOSP

def ensure_directories():

"""Ens

def update_master_database(new_data_df):

"""

if os.path.exists(MASTER_DB_PATH):

try:

## Ensure Date_Added is datetime

if 'Dat

combined_df = pd.concat([master_df, new_data_df], ignore_index=True)

excep

## Save and Encrypt

combi

audit_logger.log_action("UPDATE_MASTER", details={"records_added": len(new_data_df)})

return

def generate_hospital_sheets(master_df):

"""

if master_df.empty:

return

## Group by Hospital

group

## Simplified approach: Overwrite everything since we have the full master_df

with p

for hospital, group in grouped:

if pd.is

## Create a safe filename and sheet name

safe_

## Sort by Year

if 'Yea

## 1. Save to individual file

file_pa

## 2. Save to a sheet in Master_Database

sheet_

print(f"Generated/Updated sheet and file for {hospital_name}")

## Finally encrypt the Master Database

encryp

def update_queue_status_local(token, filename, status, details=None):

"""Loc

if token in queue:

for ite

def atomic_merge_staging_files(token):

"""

try:

with o

if token not in queue:

return

staged_items = [item for item in queue[token] if item.get('status') == 'STAGED']

if not s

results = []

maste

for item in staged_items:

filenar

if not os.path.exists(file_path):

updat

try:

## 1. L

## Check for macros (extension-based check for logging)

if filen

## 2. De-duplication

if not

if not df.empty:

## 3. M

## 4. Verify and Clean up

## If w

## Refresh master hashes for next file in loop

maste

except Exception as e:

updat

return {"results": results}

def process_new_data(new_data_df):

"""

## 2. Refresh hospital files

gener

return master_df

```

7.4 /home/team/shared/SCD_Dbase_Sorter/processor/mailer.py

```pyth

import smtplib

impor

## Configuration file path

HOSP

## SMTP Configuration (dummy defaults, can be overridden)

SMTP

```
def load_hospital_emails():
                                                                """
df = pd.read_csv(HOSPITAL_EMAILS_PATH)
                                                                return

def get_hospital_email(hospital_name):
                                                                """Ret

def get_validator_email(hospital_name):
                                                                """Ret
```

```
def get_all_hospitals():
                                                                """Ret

def get_hospital_data_path(hospital_name):
                                                                """

def load_hospital_data(hospital_name):
                                                                """

Args:
                                                                hospit

Returns:
                                                                DataF

def _create_smtp_connection():
                                                                """Cre
```

```
def _send_email(to_email, subject, body, attachments=None):
                                                                """
Args:
                                                                to_em
Returns:
                                                                bool:
msg = MIMEMultipart()
                                                                msg['
```

Attach body as plain text

```
                                                                msg.a
```

Attach files if provided

```
                                                                if atta
```

```
part = MIMEBase('application', 'octet-stream')
                                                                                part.s
```

```
try:
                                                                                serve
```

```
def send_validation_request(hospital_name, hospital_data_df=None, attachments=None):
                                                                                """
```

```
Args:
                                                                                hospit
```

```
Returns:
                                                                                bool:
```

Auto-attach hospital's Excel file if no attachments provided

if atta

subject = f"Validation Request - {hospital_name} SCD Data"

recor

A new dataset has been submitted for the following hospital and requires validation:

Hospital: {hospital_name}

Recor

Please review the attached data and confirm its accuracy by replying to this email.

If you have any questions, please contact the database administrator.

Best regards,

SCD

def send_discovery_invitation(recipient_email, discovery_link):

"""

SCD Dbase Sorter - Project Documentation

You have been invited by the SCD Database Administrator to participate in a secure Sickle Cell Disease (SCD) Data Discovery process.

This process will help identify relevant records in your local files and external email accounts to ensure our database is comprehensive and up-to-date.

To begin, please click the secure link below:

{disco

Security Notice:

* Thi

If you did not expect this request, please ignore this email or contact the administrator.

Best regards,

SCD

def send_finalized_data(hospital_name, hospital_data_df=None, attachments=None):

"""

Args:

hospit

Returns:

bool:

Auto-attach hospital's Excel file if no attachments provided

if atta

subject = f"Finalized SCD Data - {hospital_name}"

record

Please find attached the finalized SCD (Sickle Cell Disease) data for your review and records.

Hospital: {hospital_name}

Total

If you have any questions or require further assistance, please do not hesitate to reach out.

Best regards,

SCD

def send_bulk_validation_requests(hospital_data_dict):

"""

Args:

hospit

Returns:

dict: S

def send_bulk_finalized_data(hospital_data_dict):

"""

Args:

hospit

Returns:

dict: S

def configure_smtp(host=None, port=None, username=None, password=None, from_email=None, from_name=None):

"""

Args:

host: S

==========

===

Dis

def send_discovery_invitation(recipient_email, discovery_link):

"""

Args:

recipi

Returns:

bool:

You have been invited to participate in an external data discovery scan for the SCD Database System.

Purpose: To help identify Sickle Cell Disease (SCD) records that may be missing from the central database by scanning your local files and email attachments.

What happens next:

1. Clic

Secure Discovery Link: {discovery_link}

This link will expire in 7 days. Your email access is temporary and will be revoked after the scan.

If you did not expect this invitation, please ignore this email.

Best regards,

SCD

==========

===

Hel

def export_df_to_excel(df, output_path):

"""Exp

if __name__ == "__main__":

Sim

7.5 /home/team/shared/SCD_Dbase_Sorter/processor/encryption.py

```pyth

KEY_FILE = "/home/team/shared/SCD_Dbase_Sorter/data/config/.master.key"

def ensure_key():
    """Ens

def encrypt_file(file_path):
    """Enc

    key = ensure_key()
    fernet

    with open(file_path, "rb") as f:
        data =

    encrypted_data = fernet.encrypt(data)

    with open(file_path, "wb") as f:
        f.write

def decrypt_file_to_memory(file_path):
```

"""Dec

key = ensure_key()

fernet

with open(file_path, "rb") as f:

encryp

try:

decryp

```

### 7.6 /home/team/shared/SCD_Dbase_Sorter/processor/logger.py

```pyth

LOG_DIR = "/home/team/shared/SCD_Dbase_Sorter/data/logs"

AUDIT

class AuditLogger:

def __

## Avoid adding multiple handlers if already initialized

if not

def log_action(self, action, user="system", details=None):

"""

## Global instance

audit_

```

7.7 /home/team/shared/SCD_Dbase_Sorter/processor/sanitization.py

```pyth

def sanitize_value(val):

"""

## 1. Prevent Excel Formula Injection

## Cor

## 2. Prevent XSS by escaping HTML tags

val =

return val

def sanitize_dataframe(df):

"""

```

7.8 /home/team/shared/SCD_Dbase_Sorter/processor/discovery_api.py

```pyth

## Handle both relative and absolute imports for compatibility

try:

import threading

from c

def lead_initiate_request(recipient_email, recipient_phone):

"""

## Base URL should ideally be from a config or environment variable

## For

## Send invitation email

succe

if success:

updat

return token, discovery_link

def lead_bulk_initiate_request(recipients):

"""

token = initiate_discovery(email, phone)

disco

## Queue the invitation email

execu

results.append({

"emai

```
return results

def _send_invitation_worker(token, email, link):
                                                                """Wo

def get_staging_queue(token):
                                                                """

def recipient_get_context(token):
                                                                """
```

email = request.get("recipient_email", "")

phone

Mask phone for privacy if it exists

maske

Determine if identity capture is needed

If th

If they are already verified, they don't need identity capture anymore

if statu

return {

"recip

def recipient_update_identity(token, email, phone):

"""

update_data = {

"recip

If they were in INITIATED/INVITATION_SENT and updated identity,

we

def recipient_trigger_otp(token):

"""

return send_otp(request["recipient_phone"])

def recipient_verify_phone(token, code):

"""

if verify_otp(request["recipient_phone"], code):

updat

def recipient_process_local_file(token, file_name, file_content, password=None):

"""

Allow local scan if token is valid and active (or if we are in 'pre-token' phase)

if requ

ext = os.path.splitext(file_name)[1].lower()

df = p

file_like = io.BytesIO(file_content)

try:

```
if ext i
```

```
if df.empty:
```

```
return
```

Check if identity is already verified

```
is_ver
```

```
results = {
```

```
"filena
```

```
if is_verified:
```

```
maste
```

Log finding with comparison result

status

Update status but don't overwrite other scan results

local_

new_status = request["status"]

if new

update_discovery_status(token, new_status, {"local_results": local_results})

return results

def recipient_reanalyze_all_results(token):

"""

SCD Dbase Sorter - Project Documentation

```
local_results = request.get("local_results", [])
if not l
master_hashes = get_master_patient_hashes()
bot =
updated_local_results = []
for res
df = pd.DataFrame(res["data"])
if not d
updated_local_results.append(res)
update_discovery_status(token, request["status"], {"local_results": updated_local_results})
return
def recipient_start_scan(token, provider, access_token, password_map=None):
"""
```

```
request = get_discovery_request(token)
                                                                    if not
update_discovery_status(token, "SCANNING")
bot = SearchBot(provider, access_token)
                                                                    found
master_hashes = get_master_patient_hashes()
results = []
                                                                    for att
df = bot.process_attachment(att['data'], filename, password=password)
if isinstance(df, str) and df == "PASSWORD_REQUIRED":
                                                                    result
if isinstance(df, str) and df.startswith("DAMAGED_PDF"):
                                                                    result
```

```
if not df.empty:
                                                                df_an

res_item = {
                                                                "filena
```

Log finding

```
                                                                status
```

```
update_discovery_status(token, "SCAN_COMPLETED", {"email_results": results})
                                                                return

def get_final_discovered_df(token):
                                                                """
```

all_dfs = []

Aggregate analyzed local results

local_

Aggregate analyzed email results

email_

if not all_dfs:

return

final_df = pd.concat(all_dfs, ignore_index=True)

De-duplicate by Patient_ID to avoid double-entry if same patient in

multiple files

if not f

return final_df

```

### 7.9 /home/team/shared/SCD_Dbase_Sorter/processor/discovery_service.py

```pyth

# Handle both relative and absolute imports

try:

DISCOVERY_DATA_FILE = "/home/team/shared/SCD_Dbase_Sorter/data/discovery_requests.json"

def _load_requests():

if not

def _save_requests(requests):

os.ma

```
def initiate_discovery(recipient_email, recipient_phone):
                                                                """
```

```
requests[token] = {
                                                                "recip
```

```
_save_requests(requests)
                                                                audit_
```

```
def get_discovery_request(token):
                                                                """
```

```
request = requests[token]
```

Check status

```
                                                                if requ
```

Check expiry

expiry

return request

def revoke_discovery_token(token):

"""

def update_discovery_status(token, status, additional_data=None):

"""

requests[token]["status"] = status

reque

if additional_data:

reque

_save_requests(requests)

audit_

def purge_expired_requests():

"""

if len(valid_requests) != len(requests):

_save

```

### 7.10 /home/team/shared/SCD_Dbase_Sorter/processor/staging_api.py

```pyth

app = Flask(__name__)

## Manual CORS implementation since flask-cors is not available

@app

STAGING_BASE_DIR = "/home/team/shared/SCD_Dbase_Sorter/data/staging"

QUEU

def update_queue_status(token, filename, status, details=None):

os.ma

if token not in queue:

queue

found = False

for ite

if not found:

queue

with open(QUEUE_FILE, 'w') as f:

json.d

@app.route('/health', methods=['GET'])

def he

@app.route('/api/staging/init', methods=['POST', 'OPTIONS'])

def ini

data = request.json or {}

token

token_dir = os.path.join(STAGING_BASE_DIR, token)

os.ma

update_queue_status(token, "SESSION", "INITIALIZED", "Staging session started")

return jsonify({"message": "Staging initialized", "token": token}), 200

def scan_malware(file_storage):

"""

@app.route('/api/staging/upload/<token>', methods=['POST', 'OPTIONS'])

def up

if 'file' not in request.files:

return

file = request.files['file']

if file.f

## conceptual malware scan hook

if scan

```
token_dir = os.path.join(STAGING_BASE_DIR, token)
                                                                                os.ma

file_path = os.path.join(token_dir, file.filename)
                                                                                file.sa

update_queue_status(token, file.filename, "STAGED", f"Received at {datetime.datetime.now().strftime('%H:%M:%S')}")

return jsonify({
                                                                                "mess

@app.route('/api/staging/status/<token>', methods=['GET'])
                                                                                def ge

try:
                                                                                with o

return jsonify({
                                                                                "token

if __name__ == '__main__':
```

Def

```
```

7.11 /home/team/shared/SCD_Dbase_Sorter/processor/search_bot.py

```pyth

import os

impor

class SearchBot:

def __

def scan_emails(self):

"""
```

```
def _scan_gmail(self):
                                                                    from g

creds = Credentials(self.access_token)
                                                                    servic

query = " OR ".join(self.keywords)
                                                                    result

found_attachments = []
                                                                    for ms
```

```
def _scan_outlook(self):
                                                                    impor

headers = {'Authorization': f'Bearer {self.access_token}'}
```

query

```
response = requests.get(endpoint, headers=headers)
```

if resp

```
messages = response.json().get('value', [])
```

found

```
def process_attachment(self, attachment_data, filename, password=None):
```

"""

Save to temp file for processing if needed, but prefer in-memory

file_lik

```
try:
```

if ext i

def process_pdf(self, file_like):

"""

```
def identify_missing_records(self, found_df, master_db_hashes):
    """

    if 'Patient_ID' not in found_df.columns:
        return

    def is_missing(pid):
        if pd.is

    found_df['Is_Missing'] = found_df['Patient_ID'].apply(is_missing)
    return
```

7.12 /home/team/shared/SCD_Dbase_Sorter/processor/word_processor.py

```pyth
```

```
def process_word_file(file_path):
                                                                """
```

```
for table in doc.tables:
                                                                if len(t
```

Determine column mapping from the first row

```
                                                                heade
```

If no recognized headers, try the second row (handles merged title rows)

```
                                                                if not
```

Extract data if mapping was found

```
                                                                if map
```

return pd.DataFrame(all_data)

def scan_text_for_keywords(file_path, keywords=None):
"""

doc = docx.Document(file_path)
full_te

content = " ".join(full_text)
found_

```

### 7.13 /home/team/shared/SCD_Dbase_Sorter/processor/payments.py

```pyth

```
def render_paypal_button(client_id, amount="99.00", item_name="SCD Dbase Sorter - Lifetime License"):
                                                                                                    """
```

```
paypal_html = f"""
                                                                                                    <div id
```

st.markdown(f"### Upgrade to Pro")

st.writ

```

### 7.14 /home/team/shared/SCD_Dbase_Sorter/companion/scanner.py

```pyth

# --- Sanitization Protocol (Shared Logic) ---

def sa

def sanitize_dataframe(df):

for co

```
class CompanionScanner:
                                                                    def __

def log(self, message):
                                                                    print(f

def malware_check(self, file_path):
                                                                    """

def scan_folders(self):
                                                                    self.lo
```

found_files = []

for fol

return found_files

def process_and_upload(self, file_path):

filena

1. Malware Check

if self.

2. Sanitization

self.lo

if filename.lower().endswith(('.xlsx', '.xls')):

try:

3. Stream to Staging API

self.lo

def main():

print("

In a real build, these would be passed via command line or a config

file

```
API_U
```

```
if TOKEN == "DISCOVERY_TOKEN_REQUIRED":
```

```
print("
```

```
scanner = CompanionScanner(API_URL, TOKEN)
```

Init session

```
try:
```

```
files = scanner.scan_folders()
```

```
print(f
```

```
success_count = 0
```

```
for file
```

```
print(f"\nScan complete. {success_count} files successfully staged.")
```

```
print("
```

```
if sys.platform == "win32":
```

```
input(
```

```
if __name__ == "__main__":
                                                                                                    main()
```